

Rapport de TP VCL N°2

2^{ème} année Cycle Supérieur (2CS)

Option : Systèmes Informatiques (SIQ)

Thème :

Virtualisation sous Linux avec KVM

Réalisé par :

- SADAT Souhail Abdelmouaiz
- SALMA Samy

Groupe : 02

ANNEE : 2022/2023

Table des matières

Introduction :	3
Partie A : Utilisation de QEMU / KVM sous Linux	4
1. Préparation de l'environnement sous VMware Workstation :	5
1.1. Préparation des machines virtuelles :	5
1.1.1. Serveur KVM :	5
1.1.2. Client KVM :	5
1.1.3. Remarques :	5
1.2. Installation et configuration des paramètres IP	6
2. Création du réseau virtuel pour KVM	7
3. Installation de KVM	7
3.1. Installation et configuration	7
3.2. Vérification de la configuration	8
4. Création et gestion des machines virtuelles sur Client KVM	8
4.1. Création de VM1 « Ubuntu » avec virt-manager	9
4.2. Création de VM2 « RedHat » avec virt-install	11
4.3. Gestion des machines virtuelles avec virsh	12
Partie B : Utilisation de l'API libvirt avec Python	15
Conclusion :	21

Introduction :

KVM est un module de virtualisation intégré dans le noyau Linux. Considéré comme un hyperviseur de type 1, il utilise l'émulateur QEMU qui émule les architectures des processeurs des systèmes invité pour exécuter les instructions sur le système hôte. Afin de gérer la virtualisation des domaines invités par les utilisateurs, QEMU / KVM utilise l'API libvirt qui fonctionne comme un daemon sous Linux et offre des outils pour la gestion de virtualisation.

Ce TP est divisé en deux parties, dans la partie A nous avons créé des machines virtuelles sur l'hyperviseur KVM en se servant des différents outils (virt-manager, virt-install, virt-viewer, etc). Par ailleurs, dans la partie B nous avons conçu un script python basé sur la bibliothèque libvirt pour la gestion des machines virtuelles.

Partie A

Utilisation de QEMU / KVM sous Linux

1. Préparation de l'environnement sous VMware Workstation

1.1. Préparation des machines virtuelles

Pour atteindre les objectifs du TP, les machines suivantes sont créées et configurées sur VMware Workstation :

1.1.1. Serveur KVM

Cette machine a comme système d'exploitation CentOS 7.0, elle intègre le module KVM qui joue le rôle d'un hyperviseur de type 1. Sa configuration est comme suit :

Device	Summary
Memory	4 GB
Processors	2
Hard Disk (SCSI)	40 GB (Preallocated)
CD/DVD (IDE)	Using file D:\2CS\2CS_S1\VC...
Network Adapter	Custom (VMnet10)
Network Adapter 2	Custom (VMnet10)
USB Controller	Present
Sound Card	Auto detect
Printer	Present
Display	Auto detect

Virtual machine name

KVM_Server

Guest operating system

☐ Microsoft Windows

☒ Linux

☐ VMware ESX

☐ Other

Version:

CentOS 7 64-bit

1.1.2. Client KVM

Le système d'exploitation installé sur cette machine est également CentOS 7.0, elle joue le rôle de client de gestion des machine virtuelles installées sur **Serveur KVM**. Sa configuration est comme suit :

Device	Summary
Memory	1 GB
Processors	2
Hard Disk (SCSI)	10 GB (Preallocated)
CD/DVD (IDE)	Using file D:\2CS\2CS_S1\VC...
Network Adapter	Custom (VMnet10)
USB Controller	Present
Sound Card	Auto detect
Printer	Present
Display	Auto detect

Virtual machine name

KVM_Server

Guest operating system

☐ Microsoft Windows

☒ Linux

☐ VMware ESX

☐ Other

Version:

CentOS 7 64-bit

1.1.3. Remarques

- Les adaptateurs réseaux de toutes les machines sont configurés sur **Host-only** est connectés à **VMnet10** (le réseau virtuel qu'on a créé dans le TP1)
- Les processeurs des machines **Serveur KVM** et **Client KVM** doivent supporter la virtualisation, pour cela on doit leur cocher la case suivante :

Virtualization engine

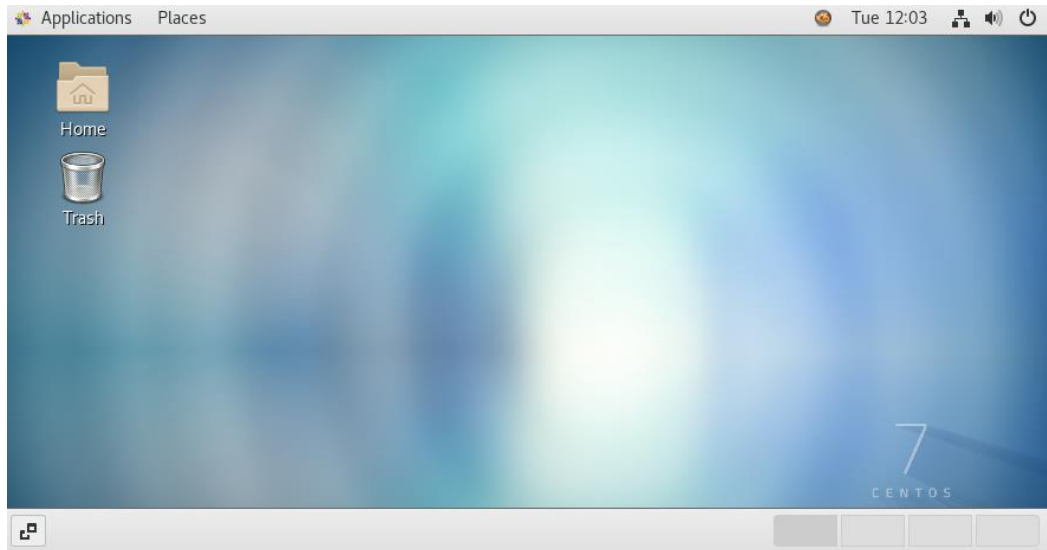
☒ Virtualize Intel VT-x/EPT or AMD-V/RVI

☐ Virtualize CPU performance counters

☐ Virtualize IOMMU (IO memory management unit)

1.2. Installation et configuration des paramètres IP

Une fois les deux machines installées, voilà comment ils s'affichent :



Les paramètres IP de la machine KVM_Server sont configurés comme suit :

A screenshot of the 'Wired' network configuration window for the 'KVM_Server' machine. The window has tabs for 'Details', 'Identity', 'IPv4', 'IPv6', and 'Security'. The 'IPv4' tab is selected. Under 'IPv4 Method', the 'Manual' radio button is selected. Under 'Addresses', the first row is filled with '10.10.0.1' for Address, '255.255.255.0' for Netmask, and '10.10.0.1' for Gateway. The 'DNS' section shows 'Automatic' as 'OFF' and the DNS server address as '10.10.0.1'. There are 'Cancel' and 'Apply' buttons at the top.

Les paramètres IP de la machine KVM_Client sont configurés comme suit :

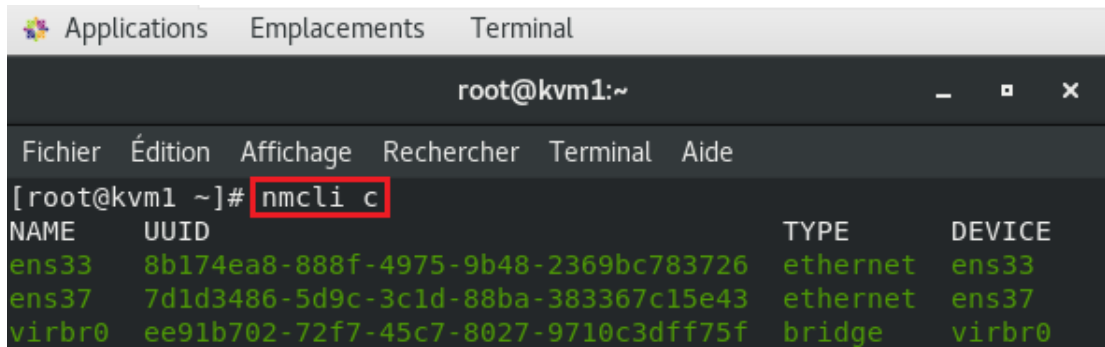
A screenshot of the 'Wired' network configuration window for the 'KVM_Client' machine. The window has tabs for 'Details', 'Identity', 'IPv4', 'IPv6', and 'Security'. The 'IPv4' tab is selected. Under 'IPv4 Method', the 'Manual' radio button is selected. Under 'Addresses', the first row is filled with '10.10.0.100' for Address, '255.255.255.0' for Netmask, and '10.10.0.1' for Gateway. The 'DNS' section shows 'Automatic' as 'OFF' and the DNS server address as '10.10.0.1'. There are 'Cancel' and 'Apply' buttons at the top.

2. Création du réseau virtuel pour KVM

Sur le Serveur KVM, exécuter le script de commandes suivant :

- **Lister les connexions réseau :**

`#nmcli c`



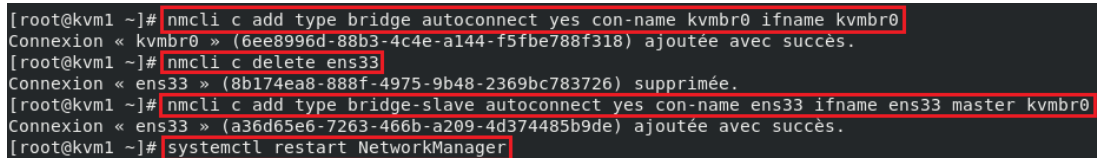
```
root@kvm1:~  
[root@kvm1 ~]# nmcli c  
NAME      UUID                                  TYPE      DEVICE  
ens33     8b174ea8-888f-4975-9b48-2369bc783726 ethernet  ens33  
ens37     7d1d3486-5d9c-3c1d-88ba-383367c15e43 ethernet  ens37  
virbr0    ee91b702-72f7-45c7-8027-9710c3dff75f bridge    virbr0
```

- **Ajouter un bridge réseau « kvmbr0 » sur l'interface « ens33 » :**

`#nmcli c add type bridge autoconnect yes con-name kvmbr0 ifname kvmbr0`

`#nmcli c delete ens33`

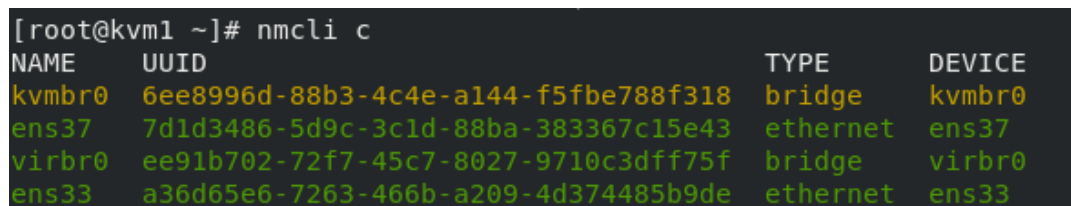
`#nmcli c add type bridge-slave autoconnect yes con-name ens33 ifname ens33 master kvmbr0`



```
[root@kvm1 ~]# nmcli c add type bridge autoconnect yes con-name kvmbr0 ifname kvmbr0  
Connexion « kvmbr0 » (6ee8996d-88b3-4c4e-a144-f5fbe788f318) ajoutée avec succès.  
[root@kvm1 ~]# nmcli c delete ens33  
Connexion « ens33 » (8b174ea8-888f-4975-9b48-2369bc783726) supprimée.  
[root@kvm1 ~]# nmcli c add type bridge-slave autoconnect yes con-name ens33 ifname ens33 master kvmbr0  
Connexion « ens33 » (a36d65e6-7263-466b-a209-4d374485b9de) ajoutée avec succès.  
[root@kvm1 ~]# systemctl restart NetworkManager
```

- **Vérifier la création du réseau virtuel :**

`#nmcli c`



```
[root@kvm1 ~]# nmcli c  
NAME      UUID                                  TYPE      DEVICE  
kvmbr0    6ee8996d-88b3-4c4e-a144-f5fbe788f318 bridge    kvmbr0  
ens37     7d1d3486-5d9c-3c1d-88ba-383367c15e43 ethernet  ens37  
virbr0    ee91b702-72f7-45c7-8027-9710c3dff75f bridge    virbr0  
ens33     a36d65e6-7263-466b-a209-4d374485b9de ethernet  ens33
```

3. Installation de KVM

3.1. Installation et configuration

Sur les machines **Serveur KVM** et **Client KVM**, exécuter le script de commandes suivant dans l'ordre :

- **Mettre à jour le système et installer les paquets : (besoin de connexion internet)**

`#yum -y update`

`#yum -y install qemu-kvm libvirt virt-install virt-viewer virt-manager`

- **Activer le module KVM :**

`#modprobe kvm`

- Démarrer le daemon de libvirt :

```
#systemctl start libvirtd
#systemctl enable libvirtd
```

3.2. Vérification de la configuration

Afin de vérifier l'installation et la configuration faite, on a utilisé les commandes suivantes :

- Vérifier que la virtualisation est activée :

```
#lscpu | grep Virtualization
```

```
[root@kvm1 ~]# lscpu | grep Virtualization
Virtualization:      VT-x
Virtualization type: full
```

- Vérifier le chargement du module KVM :

```
#lsmod | grep kvm
```

```
[root@kvm1 ~]# lsmod | grep kvm
kvm_intel             188740  0
kvm                   637289  1 kvm_intel
irqbypass             13503  1 kvm
```

- Vérifier le démarrage du service libvirt :

```
#systemctl status libvirtd
```

```
[root@kvm1 ~]# systemctl status libvirtd
● libvirtd.service - Virtualization daemon
   Loaded: loaded (/usr/lib/systemd/system/libvirtd.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2022-11-14 16:43:08 CET; 6min ago
     Docs: man:libvirtd(8)
           https://libvirt.org
   Main PID: 1393 (libvirtd)
    Tasks: 19 (limit: 32768)
   CGroup: /system.slice/libvirtd.service
           └─1393 /usr/sbin/libvirtd
             └─1851 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/defau...
               └─1852 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/defau...

Nov 14 16:43:09 kvm1 dnsmasq[1851]: using nameserver 10.10.0.1#53
Nov 14 16:43:09 kvm1 dnsmasq[1851]: read /etc/hosts - 3 addresses
Nov 14 16:43:09 kvm1 dnsmasq-dhcp[1851]: read /var/lib/libvirt/dnsmasq/default.a...es
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.542+0000: 1517: in...g)
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.542+0000: 1517: in...m1
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.542+0000: 1517: er...ry
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.544+0000: 1517: er...ry
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.544+0000: 1517: er...ry
Nov 14 16:43:09 kvm1 libvirtd[1393]: 2022-11-14 15:43:09.544+0000: 1517: er...ry
Hint: Some lines were ellipsized, use -l to show in full.
```

4. Création et gestion des machines virtuelles sur Client KVM

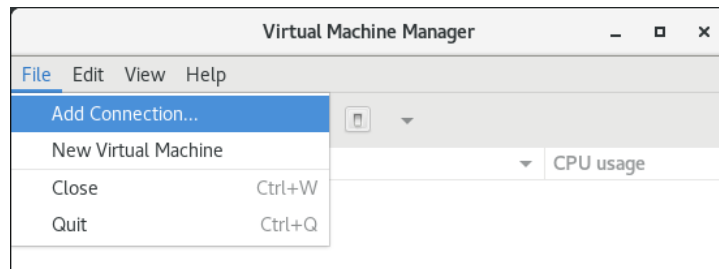
L'objectif est de créer et gérer les machines virtuelles sur l'hyperviseur KVM intégré dans la machine **Serveur KVM** à partir de la machine **Client KVM** en se connectant via SSH. Pour cela, on doit exécuter les instructions suivantes par ordre sur la machine **Client KVM** :

- Installer le paquet « openssh-askpass » :

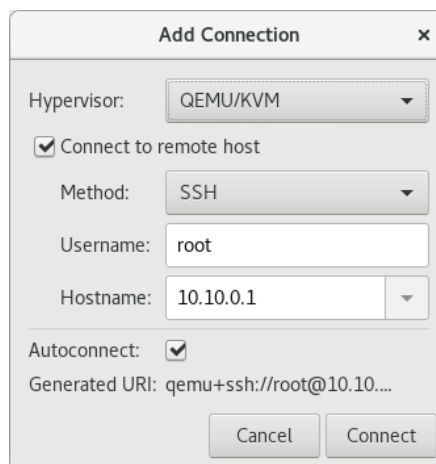
```
# yum -y install openssh-askpass
```


4.1. Création de VM1 « Ubuntu » avec virt-manager

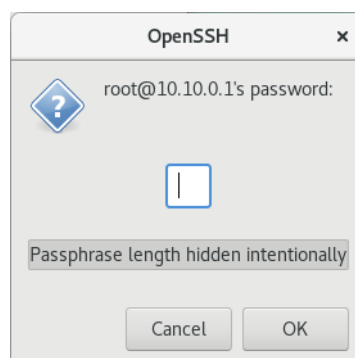
- Ouvrir l'outil virt-manager graphiquement ou par la commande :
`#virt-manager`
- Sur virt-manager, ajouter une connexion :



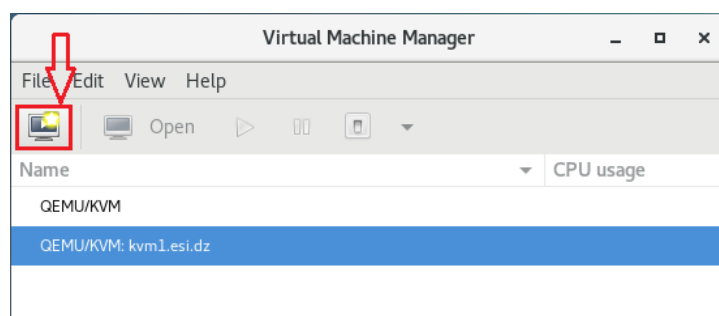
- Saisir les paramètres de la connexion SSH :



- Saisir le mot de passe du root de Serveur KVM :



- Une fois la connexion établie au Serveur KVM, ajouter une machine virtuelle :

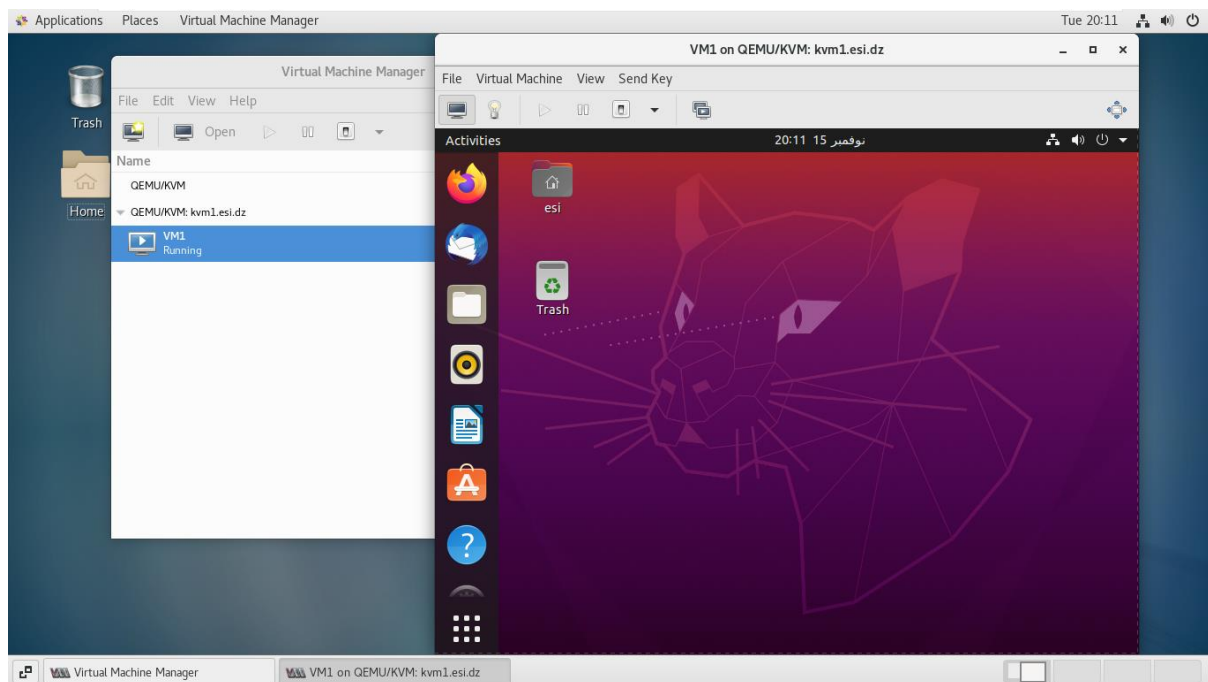


- **Utiliser une image Ubuntu avec une installation locale :**

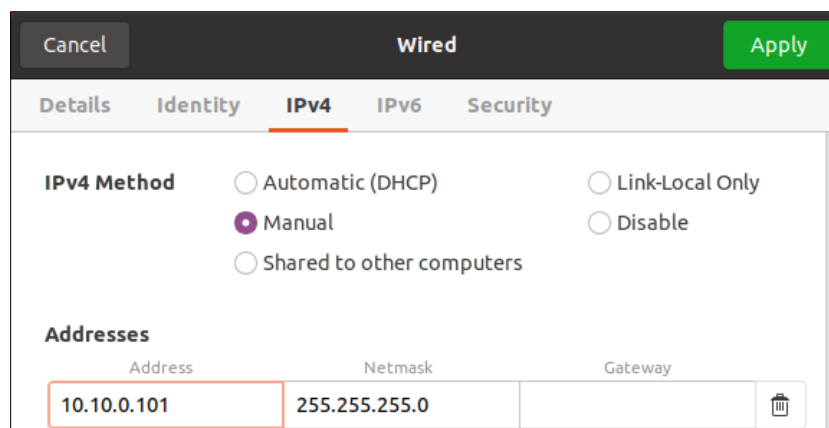
- **Suivre les étapes et créer un disque local :**

- **Sélectionner le réseau virtuel « bridge kvmbr0 » et compléter la configuration :**

- Il faut introduire une autre fois le mot de passe du root de Serveur KVM, la VM s'affiche dans une fenêtre virt-viewer, suivre les étapes pour installer Ubuntu :



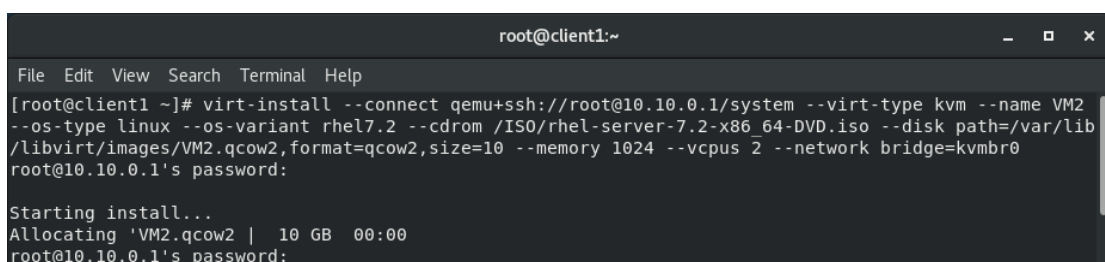
- Une fois l'installation finie, attribuer l'adresse IP « 10.10.0.101 » à VM1 :



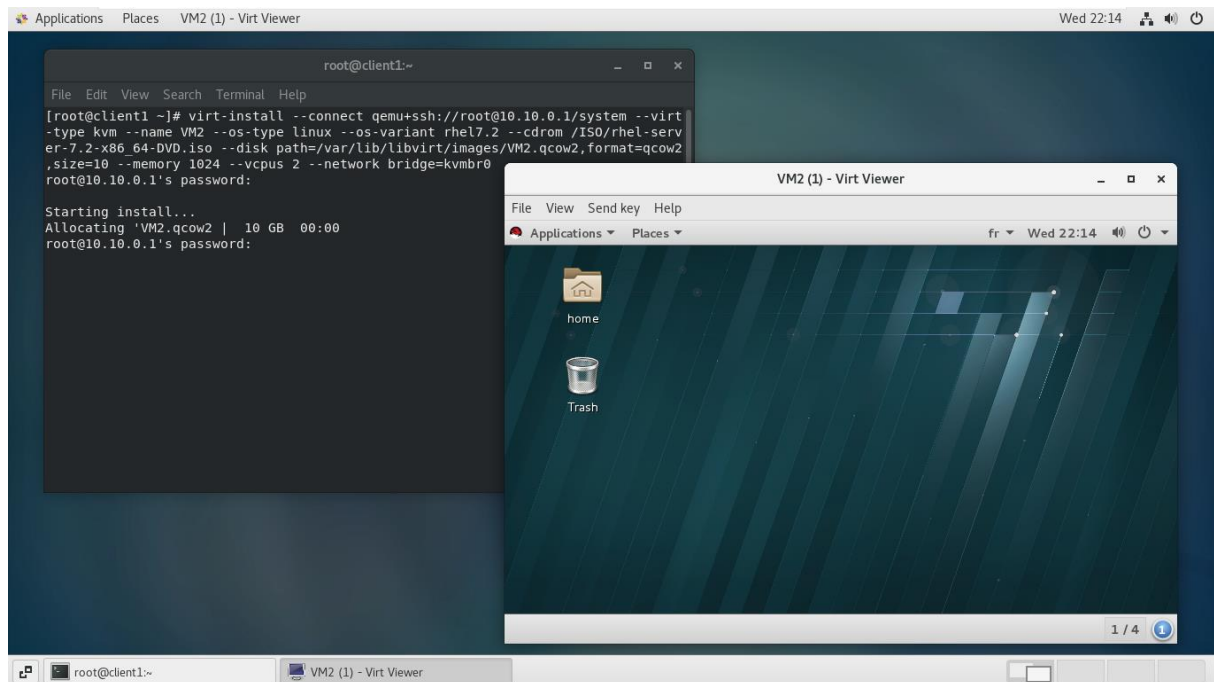
4.2. Création de VM2 « RedHat » avec virt-install

- Exécuter la commande suivante pour installer la VM :

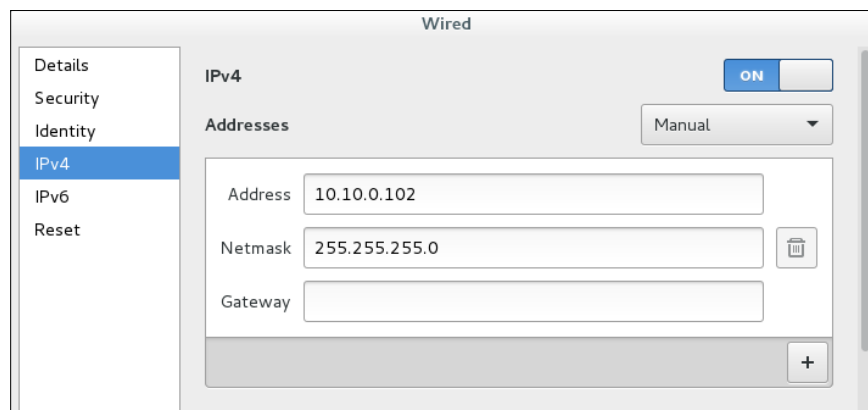
```
#virt-install --connect qemu+ssh://root@10.10.0.1/system --virt-type kvm --name VM2
--os-type linux --os-variant rhel7.2 --cdrom /ISO/rhel-server-7.2-x86_64-DVD.iso
--disk path=/var/lib/libvirt/images/VM2.qcow2,format=qcow2,size=10 --memory 1024
--vcpus 2 --network bridge=kvmbr0
```



- Introduire le mot de passe du root de Serveur KVM, la VM s'ouvre dans une fenêtre virt-viewer, suivre les étapes pour installer RedHat :

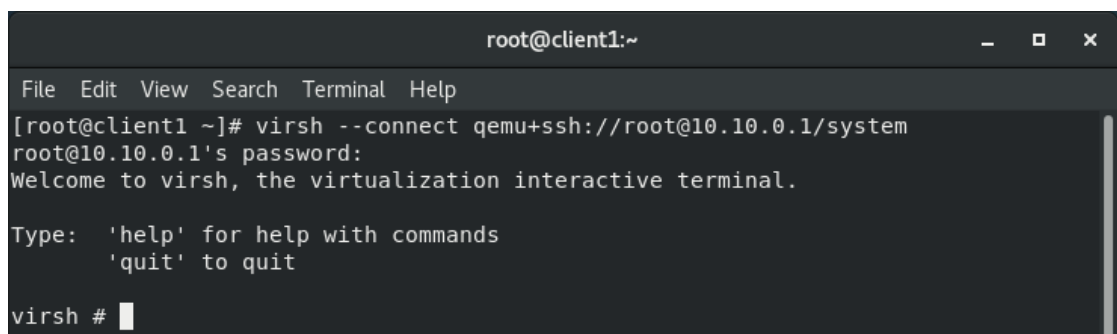


- Une fois l'installation terminée, attribuer l'adresse IP « 10.10.0.102 » à VM2 :



4.3. Gestion des machines virtuelles avec virsh

- D'abord, se connecter sur l'hyperviseur à partir de Client KVM via SSH :
#virsh --connect qemu+ssh://root@10.10.0.1/system



- Démarrer, arrêter et lister les machines virtuelles et leurs états :

```

virsh # list --all
Id      Name                               State
-----
-       VM1                                shut off
-       VM2                                shut off

virsh # start VM1
Domain VM1 started

virsh # list --all
Id      Name                               State
-----
5       VM1                                running
-       VM2                                shut off

virsh # shutdown VM1
Domain VM1 is being shutdown

virsh # list --all
Id      Name                               State
-----
-       VM1                                shut off
-       VM2                                shut off

virsh #

```

- Afficher des informations sur les domaines invités (machines virtuelles) :

Pour VM1 :

```

virsh # dominfo VM1
Id: -
Name: VM1
UUID: 7a519af1-a507-4556-b5d2-77484cf2010e
OS Type: hvm
State: shut off
CPU(s): 2
Max memory: 2097152 KiB
Used memory: 2097152 KiB
Persistent: yes
Autostart: disable
Managed save: no
Security model: selinux
Security DOI: 0

virsh # domstats VM1
Domain: 'VM1'
state.state=5
state.reason=1
balloon.maximum=2097152
vcpu.current=2
vcpu.maximum=2
block.count=2
block.0.name=vda
block.0.path=/var/lib/libvirt/images/VM1.qcow2
block.0.allocation=10739326976
block.0.capacity=10737418240
block.0.physical=10739318784
block.1.name=hda

virsh # domiflist VM1
Interface Type      Source      Model      MAC
-----
-         bridge    kvmbr0      virtio     52:54:00:17:4d:d2

```

Pour VM2 :

```
virsh # dominfo VM2
Id: -
Name: VM2
UUID: 6716241e-ac81-445d-a2e0-84f491ba4ccd
OS Type: hvm
State: shut off
CPU(s): 2
Max memory: 1048576 KiB
Used memory: 1048576 KiB
Persistent: yes
Autostart: disable
Managed save: no
Security model: selinux
Security DOI: 0

virsh # domstats VM2
Domain: 'VM2'
  state.state=5
  state.reason=1
  balloon.maximum=1048576
  vcpu.current=2
  vcpu.maximum=2
  block.count=2
  block.0.name=vda
  block.0.path=/var/lib/libvirt/images/VM2.qcow2
  block.0.allocation=3373789184
  block.0.capacity=10737418240
  block.0.physical=10739318784
  block.1.name=hda

virsh # domiflist VM1
Interface  Type          Source      Model      MAC
-----
-          bridge      kvmbr0      virtio      52:54:00:17:4d:d2
```

- Afficher des informations sur l'hyperviseur et le host:

```
virsh # version
Compiled against library: libvirt 4.5.0
Using library: libvirt 4.5.0
Using API: QEMU 4.5.0
Running hypervisor: QEMU 1.5.3

virsh # uri
qemu+ssh://root@10.10.0.1/system

virsh # hostname
kvml.esi.dz

virsh # nodeinfo
CPU model:      x86_64
CPU(s):         2
CPU frequency:  1800 MHz
CPU socket(s):  2
Core(s) per socket: 1
Thread(s) per core: 1
NUMA cell(s):   1
Memory size:    3861292 KiB

virsh # maxvcpus
240
```

Partie B

Utilisation de l'API libvirt avec Python

Remarques :

- Le script python est conçu pour être exécuté directement sur **Serveur KVM** et non pas sur **Client KVM**
- Le script est écrit en langage **Python 2.7.5** et utilise les bibliothèques **libvirt** et **os**
- Vous pouvez consulter le script avec indentation sur le lien suivant : [SADAT.SALMA.SIQ2.TP2.Script](#)

```

1 #!/usr/bin/env python2
2 # -*- coding: utf-8 -*-
3
4 import libvirt
5 import os
6
7 ''' *****
8 Déclaration des fonctions et des structures de données
9 ***** '''
10
11 #Fonction qui liste les choix du menu principal
12 def listeChoix():
13     print("      0) Informations sur la machine hyperviseur")
14     print("      1) Lister les machines virtuelles")
15     print("      2) Informations sur une machine virtuelle")
16     print("      3) Opérations sur une machine virtuelle")
17     print("      4) Visualiser une machine virtuelle")
18     print("      5) Quitter")
19
20 #Dictionnaire qui contient les opérations possibles sur une machine virtuelle
21 operations = {
22     1 : "démarrer",
23     2 : "arrêter",
24     3 : "forcer l'arrêt",
25     4 : "suspendre",
26     5 : "reprendre"
27 }
28 #Dictionnaire qui contient les états possibles d'une machine virtuelle
29 state = {
30     libvirt.VIR_DOMAIN_RUNNING : "en cours d'exécution",
31     libvirt.VIR_DOMAIN_PAUSED : "mise en pause",
32     libvirt.VIR_DOMAIN_SHUTDOWN : "en arrêt",
33     libvirt.VIR_DOMAIN_SHUTOFF : "éteinte"
34 }
35
36 #Fonction qui donne l'état d'une machine virtuelle
37 def etat(n):
38     try:
39         return state.get(n)
40     except:
41         return "éteinte"
42
43 ''' *****
44 Ne pas afficher les messages d'erreur de libvirt sur la console
45 ***** '''
46
47 #définir une fonction de gestion d'erreurs qui ne fait rien
48 def libvirt_callback(userdata, err):
49     pass
50
51 #remplacer la fonction de gestion d'erreurs de libvirt qui affichent les messages
52 #d'erreur sur la console par une fonction qui ne fait rien afin de ne pas les afficher
53 libvirt.registerErrorHandler(f=libvirt_callback, ctx=None)
54
55 ''' *****
56 ouvrir une connexion à l'hyperviseur
57 ***** '''
58
59 #spécifier l'URI de connexion (dans notre cas une connexion locale en mode système)
60 conn = libvirt.open("qemu:///system")
61 if not conn:
62     # si la connexion a échoué, afficher un message d'erreur et terminer le programme
63     raise SystemExit("La connexion à l'hyperviseur a échoué !")
64
65 ''' *****
66 Boucle du menu principal
67 ***** '''
68
69 #afficher la liste des choix du menu principal
70 print("Programme de gestion des machines virtuelles ; veuillez entrer votre choix")
71 listeChoix()
72
73 choix = -1 #initialiser le choix

```



```

74 while (choix != 5): # Reboucler jusqu'à ce que le choix soit '5'
75
76     try:
77         choix = int(raw_input("\nVotre choix : ")) #lire le choix entré par l'utilisateur
78     except:
79         choix = -1 #si le choix est erroné
80
81     if(choix == 0): # si choix = 0 alors appeler les fonctions de libvirt qui affichent
82                     # des informations sur l'hyperviseur et le host
83
84         print("\n      Informations sur la machine hyperviseur :")
85         print("      -----")
86         print("      Nom de la machine hyperviseur : "+str(conn.getHostname()))
87         print("      Nombre maximum de processus virtuels : "+str(conn.getMaxVcpus(None)))
88         # info() retourne un tuple qui contient plusieurs informations
89         print("      Modèle du processeur : "+str(conn.getInfo()[0]))
90         print("      Taille de la RAM : "+str(conn.getInfo()[1])+" Mo")
91         print("      Nombre de processeurs actifs : "+str(conn.getInfo()[2]))
92         print("      Fréquence du processeur : "+str(conn.getInfo()[3])+" MHz")
93         print("      Nombre de noeuds NUMA : "+str(conn.getInfo()[4]))
94         print("      Nombre de processeurs : "+str(conn.getInfo()[5]))
95         print("      Nombre de coeurs par processeurs : "+str(conn.getInfo()[6]))
96         print("      Nombre de threads par coeur : "+str(conn.getInfo()[7]))
97
98     elif(choix == 1): #Si le choix est 1, alors afficher la liste des machines virtuelles
99
100        #récupère toutes les machines virtuelles
101        list_VM = conn.listAllDomains()
102        #Longueur du plus long nom de machine virtuelle pour contrôler l'affichage
103        if len(list_VM) == 0:
104            max_len = 0
105        else:
106            max_len = len(max(list_VM, key = lambda d : len(d.name())).name())
107        #constituer l'entête de la liste des VM
108        print("\n      Liste des machines virtuelles :\n")
109        espace = min(max(max_len,20),20)
110        nom = "Nom".ljust(espace)
111        head = " Id      "+nom+"      État              "
112        print("      "+head)
113        print("      "+" ".rjust(len(head),'-'))
114        #Parcourir les machines virtuelles et les afficher dans la liste
115        for dom in list_VM :
116            id = str(dom.ID())
117            #Si la machine est éteinte, le ID sera -1, alors afficher le caractère '-'
118            if dom.ID() == -1:
119                id = '-'
120            print("      "+id.ljust(6)+dom.name().ljust(espace+4)+etat(dom.state()[0]))
121
122
123    elif(choix == 2): #Si le choix est 2, afficher les informations sur une VM
124
125        #Lire le nom de la machine virtuelle depuis la console
126        nom = raw_input("      Introduire le nom d'une machine pour afficher"
127                        "ses informations : ")
128
129        try:
130            #rechercher la VM par nom
131            dom = conn.lookupByName(nom)
132
133        except libvirt.libvirtError as e:
134            #Si la VM n'existe pas, afficher un message d'erreur
135            print("      Erreur : Aucun domaine invité ne porte ce nom !")
136
137        else: # si La machine virtuelle existe, alors afficher ses
138            # informations en utilisant des fonctions de libvirt
139
140            print("\n      Informations sur la machine '"+nom+"' :")
141            print("      -----"+"".ljust(len(nom),'-'))
142            print("      ID : "+str(dom.ID()))
143            # la fonction info() retourne un tuple contenant plusieurs informations
144            print("      État : "+etat(dom.info()[0]))
145            print("      Maximum Mémoire RAM : "+str(dom.info()[1]/1024)+" Mo")
146            print("      Mémoire RAM utilisée : "+str(dom.info()[2]/1024)+" Mo")

```

```

147     print("          Nombre de CPUs virtuels : "+str(dom.info()[3]))
148     print("          Temps CPU : "+str(dom.info()[4])+" ns")
149
150     if not dom.isActive(): # Si la machine est éteinte, alors il n'est pas
151                             # possible de récupérer ses adresses IP
152         print("          Adresse IP : indisponible !"
153               " Veuillez allumer la machine")
154
155     else: #Si la machine est allumée, alors afficher ses adresses IP
156
157         iffaces = {} # initialiser la structure qui contiendra les adresses
158         try:
159             # Interroger "qemu-guest-agent" qui doit être installé et configuré
160             # sur la machine virtuelle, s'il ne l'est pas, une exception sera
161             # lancée et doit être capturée dans le bloc try except
162             iffaces = dom.interfaceAddresses(
163                 libvirt.VIR_DOMAIN_INTERFACE_ADDRESSES_SRC_AGENT, 0
164             )
165
166         except libvirt.libvirtError as e:
167             # si Qemu Guest Agent n'est pas installé sur la machine virtuelle
168             # ou s'il n'est pas connecté, alors signaler ceci par le message
169             # d'erreur par défaut de libvirt
170             try:
171                 print("          Adresse IP : indisponible !"
172                       + (str(e)).split(":")[1])
173             except:
174                 print("          Adresse IP : indisponible ! ")
175
176         else: # sinon parcourir la structure retournée et extraire les adresses
177             # IP, la structure est un dictionnaire ayant comme éléments
178             # les interfaces réseaux de la machine
179
180             adresse = "" #initialiser une chaîne pour concaténer les adresses IP
181             for (name, val) in iffaces.iteritems(): #pour chaque interface réseau
182                 # extraire les champs d'adresses qui ne sont pas de type loopback
183                 if val['addrs'] and name!='lo':
184                     # Pour chaque adresse de l'interface réseau
185                     for ipaddr in val['addrs']:
186                         # extraire uniquement les adresses de type IPv4
187                         if ipaddr['type'] == libvirt.VIR_IP_ADDR_TYPE_IPV4:
188                             # concaténer l'adresses à la chaîne
189                             adresse += ipaddr['addr'] + " "
190
191             if adresse == "":
192                 # si le résultat est vide, les adresses IP sont indisponibles
193                 print("          Adresse IP : indisponible ! En cours de"
194                       " recherche ou adresses non configurées")
195             else:
196                 # afficher les adresses IP
197                 print("          Adresse IP : "+str(adresse))
198
199     elif(choix == 3): # Si le choix est 3, alors afficher un menu des opérations
200                     # possibles à effectuer sur une machine virtuelle
201         print("          Choisir l'opération à effectuer sur une machine virtuelle :)")
202         print("          1) Démarrer")
203         print("          2) Arrêter")
204         print("          3) Forcer l'arrêt")
205         print("          4) Suspendre")
206         print("          5) Reprendre")
207         print("          6) Retour")
208         try:
209             # Récupérer le numéro de l'opération choisie
210             op = int(raw_input("          Opération : "))
211         except:
212             # Traiter le cas d'un numéro erroné
213             op = -1
214         if(op in {1,2,3,4,5}):
215             # Si le numéro introduit est valide, lire le nom de la machine virtuelle
216             nom = raw_input("          Introduire le nom de la Machine que vous "
217                             "voulez " + operations.get(op) + " : ")
218             try:
219                 # Rechercher la machine virtuelle par nom
220                 dom = conn.lookupByName(nom)

```

```

220 except:
221     # Si le nom n'existe pas, afficher un message d'erreur
222     print("          Erreur : Aucun domaine invité ne porte ce nom !")
223 else:
224
225     if(op == 1): # L'opération : Démarrer une machine
226         try:
227             if(dom.state()[0]==libvirt.VIR_DOMAIN_SHUTOFF):
228                 # Démarrer la machine seulement si elle est éteinte
229                 dom.create()
230                 print("          Machine démarrée avec succès")
231                 print("          Pour visualiser la machine appuyer sur 4"
232                     " dans le menu principal")
233             else:
234                 # Sinon, afficher un message d'erreur
235                 print("          Erreur : La machine est déjà en cours"
236                     " d'exécution !")
237             except libvirt.libvirtError as e:
238                 # Afficher le message d'erreur d'une quelconque exception
239                 print("          Erreur : "+str(e))
240
241     elif(op == 2): # L'opération : Arrêter une machine
242         try:
243             if(dom.state()[0]==libvirt.VIR_DOMAIN_RUNNING):
244                 # Arrêter la machine seulement si elle est allumée
245                 dom.shutdown()
246                 print("          Machine en cours d'arrêt")
247             else:
248                 # Sinon, afficher un message d'erreur
249                 print("          Erreur : La machine n'est pas en cours"
250                     " d'exécution !")
251             except libvirt.libvirtError as e:
252                 # Afficher le message d'erreur d'une quelconque exception
253                 print("          Erreur : "+str(e))
254
255     elif(op == 3): # L'opération : Forcer l'arrêt d'une machine
256         try:
257             if(dom.state()[0]!=libvirt.VIR_DOMAIN_SHUTOFF):
258                 # Forcer l'arrêt seulement si la machine n'est pas éteinte
259                 dom.destroy()
260                 print("          Machine arrêtée avec succès")
261             else:
262                 # Sinon, afficher un message d'erreur
263                 print("          Erreur : La machine est déjà éteinte !")
264             except libvirt.libvirtError as e:
265                 # Afficher le message d'erreur d'une quelconque exception
266                 print("          Erreur : "+str(e))
267
268     elif(op == 4): # L'opération : Mettre la machine en pause
269         try:
270             if(dom.state()[0]!=libvirt.VIR_DOMAIN_SHUTOFF):
271                 # Suspendre la machine seulement si elle n'est pas éteinte
272                 dom.suspend()
273                 print("          Machine mise en pause avec succès")
274             else:
275                 # Sinon, afficher un message d'erreur
276                 print("          Erreur : La machine est éteinte !")
277             except libvirt.libvirtError as e:
278                 # Afficher le message d'erreur d'une quelconque exception
279                 print("          Erreur : "+str(e))
280
281     elif(op == 5): # L'opération : Reprendre une machine suspendue
282         try:
283             if(dom.state()[0]==libvirt.VIR_DOMAIN_PAUSED):
284                 # Reprendre la machine seulement si elle était en pause
285                 dom.resume()
286                 print("          Machine reprise avec succès")
287                 print("          Pour visualiser la machine appuyer sur 4"
288                     " dans le menu principal")
289             else:
290                 # Sinon, afficher un message d'erreur
291                 print("          Erreur : La machine n'est pas en pause !")
292             except libvirt.libvirtError as e:

```

```

293             # Afficher le message d'erreur d'une quelconque exception
294             print("          Erreur : "+str(e))
295
296         elif (op != 6): # Le choix n'est pas valide
297             print("          choix erroné !")
298
299     elif (choix == 4): # Si le choix est 4, alors ouvrir l'écran de la machine virtuelle
300         # à l'aide de l'outil virt-viewer
301
302     # Lire de la console le nom de la machine virtuelle à visualiser
303     nom = raw_input("          Introduire le nom de la Machine virtuelle à visualiser : ")
304     try:
305         # Rechercher si nom de la machine virtuelle existe
306         dom = conn.lookupByName(nom)
307     except:
308         # Si le nom n'existe pas, afficher un message d'erreur
309         print("          Erreur : Aucun domaine invité ne porte ce nom !")
310     else:
311         try:
312             # Si le nom existe, lancer une commande système qui ouvre virt-viewer
313             os.system("virt-viewer "+nom+" &")
314         except:
315             # Afficher le message d'erreur si c'est le cas
316             print("          Erreur : "+str(e))
317
318     elif (choix != 5): #Le choix introduit dans le menu principal n'est pas valide
319
320         print("Choix erroné ! veuillez entrer votre choix")
321         listeChoix()
322
323     if (choix in [0,1,2,3,4]): # Re-Afficher le menu principal
324
325         wait = raw_input("\nAppuyer sur 'Enter' pour continuer...")
326         print("\nMenu principal ; veuillez entrer votre choix")
327         listeChoix()
328
329
330 conn.close() # Fermer la connexion avec l'hyperviseur

```

Conclusion :

Nous avons appris, à travers ce TP, à mettre en place un environnement virtuel en utilisant l'hyperviseur kvm/qemu sous linux. Pour cela, nous avons utilisés plusieurs outils basés sur le daemon libvirt, soit en mode graphique (Virt-Manager et Virt-Viewer) ou en ligne de commandes (virt-install). Nous avons même créé notre propre outil à la base du module Python libvirt offert par l'API libvirt.

Nous avons ainsi acquis quelques compétences pratiques en matière de virtualisation, en espérant les approfondir encore davantage dans les prochains TPs.